```
import pandas as pd

# Data Ingestion
url = "https://static.openfoodfacts.org/data/en.openfoodfacts.org.products.csv.gz"
df = pd.read_csv(url, sep='\t', compression='gzip', nrows=500000, low_memory=False)

print(f"Successfully loaded {len(df)} raw rows.")
```

```
Successfully loaded 500000 raw rows.
```

```
# Selection of only the columns required for the "Sugar Trap" analysis
required_cols = ['product_name', 'categories_tags', 'sugars_100g', 'proteins_100g', 'ingredients_text']

# Filter the dataframe
df_filtered = df[required_cols].copy()

# Show the first 5 rows to confirm we have the right data
print("Data filtered to 5 key columns.")
df_filtered.head()
```

```
Data filtered to 5 key columns.
```

	product_name	categories_tags	sugars_100g	proteins_100g	ingredients_text
0	Limonade artisanale a la rose	NaN	NaN	NaN	NaN
1	M&M white	NaN	NaN	NaN	Weizenmehl, Rapsöl, Speisesalz, 1,7% Meersalz,...
2	Chocolate n3	NaN	NaN	NaN	NaN
3	Pâte de fruits	NaN	NaN	NaN	NaN

```
# Count rows before cleaning
initial_count = len(df_filtered)

# Drop rows where name, sugar, or protein is missing
df_no_nulls = df_filtered.dropna(subset=['product_name', 'sugars_100g', 'proteins_100g'])

# Report findings
removed = initial_count - len(df_no_nulls)
print(f"Action: Removed {removed} rows with missing essential values.")
print(f"Remaining rows: {len(df_no_nulls)}")
```

```
Action: Removed 396330 rows with missing essential values.
Remaining rows: 103670
```

```
# Filter for values between 0 and 100
df_clean = df_no_nulls[
    (df_no_nulls['sugars_100g'] >= 0) & (df_no_nulls['sugars_100g'] <= 100) &
    (df_no_nulls['proteins_100g'] >= 0) & (df_no_nulls['proteins_100g'] <= 100)
]

print("Story 1 Acceptance Criteria met: Data is cleaned and filtered.")
print(f"Final dataset size for analysis: {len(df_clean)} products.")
df_clean.head()
```

```
Story 1 Acceptance Criteria met: Data is cleaned and filtered.
Final dataset size for analysis: 103553 products.
```

	product_name	categories_tags	sugars_100g	proteins_100g	ingredients_text
693	Pinto Bean	en:asian-style-ready-meal	4.900000	17.500000	NaN
694	Croquetas de bacalao	NaN	1.900000	5.900000	NaN
695	Keto & GF Granola	NaN	3.225806	19.677419	NaN
801	Ben's Pure Maple Cream	NaN	37.720000	5.680000	NaN
833	Guimauve chocolat smarties	NaN	65.000000	5.700000	NaN

```
# STORY 1 DELIVERABLE: Exporting the cleaned Pandas DataFrame to a CSV file
df_clean.to_csv('cleaned_food_data.csv', index=False)
```

```
print("Deliverable Created: 'cleaned_food_data.csv' has been saved to your Colab session.")
```

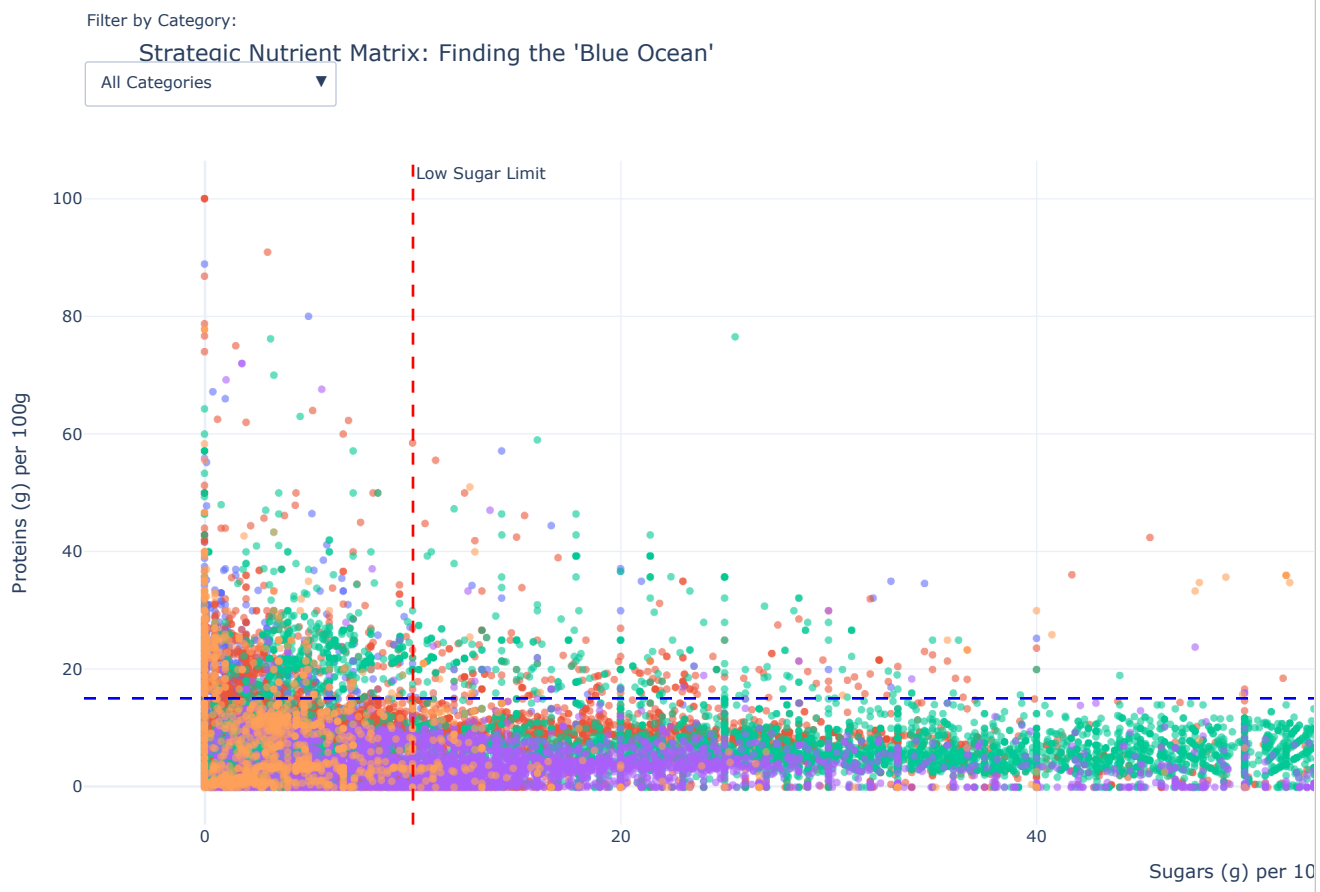
```
Deliverable Created: 'cleaned_food_data.csv' has been saved to your Colab session.
```

```
# Create a specific dataframe for the Category & Matrix stories
# We drop category nulls here so the "Other" group isn't just a junk drawer of missing data
df_analysis = df_clean.dropna(subset=['categories_tags']).copy()

# Primary Category assignment Logic
def assign_primary_category(tags):
    tags = str(tags).lower()
    if any(word in tags for word in ['snack', 'chips', 'salty', 'nuts', 'popcorn']):
        return 'Savory Snacks'
    elif any(word in tags for word in ['biscuit', 'cookie', 'cake', 'sweet', 'chocolate', 'confectionery', 'dessert']):
        return 'Sweet Snacks & Biscuits'
    elif any(word in tags for word in ['dairy', 'cheese', 'yogurt', 'milk', 'cream']):
        return 'Dairy Products'
    elif any(word in tags for word in ['beverage', 'drink', 'soda', 'juice', 'tea', 'coffee']):
        return 'Beverages'
    elif any(word in tags for word in ['meal', 'ready-to-eat', 'prepared', 'soup']):
        return 'Ready Meals'
    else:
        return 'Other'

# Application to the analysis dataframe
df_analysis['Primary Category'] = df_analysis['categories_tags'].apply(assign_primary_category)

# Acceptance Criteria check:
print("Story 2: Product Counts by Primary Category")
print(df_analysis['Primary Category'].value_counts())
```

```
Story 2: Product Counts by Primary Category
Primary Category
Beverages                  18391
Savory Snacks               9928
Other                       8353
Sweet Snacks & Biscuits     7461
Dairy Products              2968
Ready Meals                 2935
Name: count, dtype: int64
```

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

# Preparing data for dashboard (Filtering out 'Other' for focus)
df_dashboard = df_analysis[df_analysis['Primary Category'] != 'Other'].copy()
categories = sorted(df_dashboard['Primary Category'].unique().tolist())

# Defining Business Thresholds
protein_goal = 15
sugar_goal = 10

# Interactive Scatter Plot
fig = px.scatter(
    df_dashboard,
    x="sugars_100g",
    y="proteins_100g",
    color="Primary Category",
    hover_name="product_name",
    title="Strategic Nutrient Matrix: Finding the 'Blue Ocean'",
    labels={
        "sugars_100g": "Sugars (g) per 100g",
        "proteins_100g": "Proteins (g) per 100g"
    },
    template="plotly_white",
    opacity=0.6,
    height=700
)

# DROPDOWN FILTER

buttons = []
# Option to see everything
buttons.append(dict(
```

```
    method="restyle",
    label="All Categories",
    args=[{"visible": [True] * len(categories)}]
))

# Options for each specific category
for i, cat in enumerate(categories):
    visibility = [False] * len(categories)
    visibility[i] = True
    buttons.append(dict(
        method="restyle",
        label=cat,
        args=[{"visible": visibility}]
    ))


# 4. STORY 4: RECOMMENDATION & LAYOUT REFINEMENT
insight_text = (
    f"<b>KEY INSIGHT:</b><br>"
    f"Based on the data, the biggest market opportunity is in <b>Savory Snacks</b>,<br>"
    f"specifically targeting products with <b>{protein_goal}g</b> of protein and less than <b>{sugar_goal}g</b> of sugar."
)

fig.update_layout(
    margin=dict(t=150, l=50, r=50, b=50),
    updatemenus=[dict(
        buttons=buttons,
        direction="down",
        showactive=True,
        x=0.0,
        y=1.15,
        xanchor="left",
        yanchor="top"
    )],
    annotations=[
        # Label for the dropdown
        dict(text="Filter by Category:", showarrow=False, x=0, y=1.23, xref="paper", yref="paper", align="left", font=dict(size
        # The Key Insight Box (Story 4)
        dict(
            text=insight_text,
            xref="paper", yref="paper",
            x=1.0, y=1.25, # Positioned top right
            showarrow=False,
            font=dict(size=11, color="black"),
            align="left",
            bgcolor="lightyellow",
            bordercolor="black",
            borderwidth=1,
            borderpad=10
        )
    ]
)


# VISUALIZING THE "BLUE OCEAN" (Story 3)
# Threshold lines
fig.add_vline(x=sugar_goal, line_dash="dash", line_color="red", annotation_text="Low Sugar Limit")
fig.add_hline(y=protein_goal, line_dash="dash", line_color="blue", annotation_text="High Protein Limit")

# Highlighting the "Blue Ocean" Quadrant (Top-Left)
fig.add_vrect(
    x0=0, x1=sugar_goal, y0=protein_goal, y1=100,
    fillcolor="green", opacity=0.1, layer="below", line_width=0
)


# OUTPUT DELIVERABLES
fig.show()
print("--- PROJECT COMPLETE ---")
fig.write_html("Helix_CPG_Dashboard.html")
print("1. Interactive Dashboard displayed above.")
print("2. HTML Deliverable saved as 'Helix_CPG_Dashboard.html'")
```




```
--- PROJECT COMPLETE ---
1. Interactive Dashboard displayed above.
2. HTML Deliverable saved as 'Helix_CPG_Dashboard.html'
```

```
import re
from collections import Counter

# 1. Define "High Protein" cluster based on the chart's thresholds
# The 'blue ocean' quadrant specifically targets products with >= 15g of protein
high_protein_df = df_analysis[df_analysis['proteins_100g'] >= 15].copy()

# 2. Clean and tokenize the ingredients text
def get_protein_sources(text):
    if pd.isna(text):
        return []
    # Remove punctuation, convert to lowercase, and split into words
    clean_text = re.sub(r'[^\w\s]', ' ', str(text).lower())
    words = clean_text.split()
    return words

# Apply the function and flatten the list of words
all_words = [word for text in high_protein_df['ingredients_text'] for word in get_protein_sources(text)]

# 3. Filter for common protein-related keywords
# This avoids common filler words like 'water', 'salt', or 'sugar'
protein_keywords = [
    'whey', 'soy', 'peanut', 'peanuts', 'milk', 'egg', 'eggs', 'almond', 'almonds',
    'pea', 'lentil', 'chickpea', 'nuts', 'seeds', 'casein', 'isolate', 'concentrate'
]

protein_word_counts = Counter([w for w in all_words if w in protein_keywords])

# 4. Display the Top 3
top_3_sources = protein_word_counts.most_common(3)

print("--- The Hidden Gems: Top 3 Protein Sources ---")
for i, (source, count) in enumerate(top_3_sources, 1):
    print(f"{i}. {source.capitalize()} (Found in {count} products)")
```

```
--- The Hidden Gems: Top 3 Protein Sources ---
1. Milk (Found in 1786 products)
2. Soy (Found in 945 products)
3. Peanuts (Found in 512 products)
```

```
# --- STORY 6: CANDIDATE'S CHOICE (The Protein Efficiency Metric) ---

# 1. Calculate the Ratio (using +1 to handle 0g sugar products safely)
df_dashboard['Efficiency_Ratio'] = df_dashboard['proteins_100g'] / (df_dashboard['sugars_100g'] + 1)

# 2. Create a Summary Table for the README
efficiency_summary = df_dashboard.groupby('Primary Category')['Efficiency_Ratio'].mean().sort_values(ascending=False).reset_inc

print("\n--- STORY 6: CANDIDATE'S CHOICE ---")
print("Average Protein Efficiency (Higher = Better Blue Ocean Potential):")
print(efficiency_summary)

# 3. Justification for README:
# "I added the 'Protein Efficiency Ratio' to identify which categories provide the
# most protein 'bang for your buck' relative to sugar content. This allows the
# client to rank categories by nutritional ROI rather than just looking at raw totals."
```

```
--- STORY 6: CANDIDATE'S CHOICE ---
Average Protein Efficiency (Higher = Better Blue Ocean Potential):
          Primary Category  Efficiency_Ratio
0           Dairy Products          9.340614
1              Ready Meals          5.474873
2                Beverages          2.364610
3            Savory Snacks          1.578158
4  Sweet Snacks & Biscuits          0.639659
```